

COMP309 DATAWAREHOUSING AND PREDICTIVE ANALYTICS
(Section 001)

GROUP PROJECT

CITY OF TORONTO'S PREDICTIVE MODEL ON BICYCLE THEFTS

Group 4:
Thanh Tam Dang
Thu Ha Le
Xuan Tri Nguyen
Hetvi Prajapati

Centennial College
Fall 2025

Table of Contents

1. Executive Summary	3
2. Overview of the Solution.....	3
3. Data Exploration and Findings.....	4
3.1. Dataset Summary	4
3.2. Column Types	4
3.3. Key Data Quality Findings	4
3.4. Visualizations Included	5
4. Feature Selection	10
5. Data Modeling.....	12
5.1 Data Cleaning Strategy	12
5.2 Data Wrangling Techniques.....	13
5.3 Assumptions and Constraints.....	13
5.4 Train-Test Split.....	14
5.5 Preprocessing Pipelines.....	14
5.6 Feature Selection	14
5.7 Handling Class Imbalance.....	15
6. Predictive Model Building and Evaluation.....	15
6.1 Logistic Regression.....	15
6.2 Decision Tree Classifier	16
6.3 ROC Curve Comparison	16
6.4 Best Model Selection	17
7. Deployment of the Model.....	18
7.1. Model Serialization.....	18
7.2. Flask-Based API Service.....	18
7.3. Preprocessing and Prediction Flow	18
7.4. HTML User Interface	19
7.5. Testing the API (Postman + UI).....	19
7.6. Deployment Output Summary	23

1. Executive Summary

This project analyzes the Toronto Police Bicycle Theft Open Data to determine whether a stolen bicycle is likely to be recovered. The dataset includes more than 39,000 theft incidents from 2013–2025, with detailed attributes such as occurrence dates, bike characteristics, locations, and theft status.

The primary goal of this project was to build a predictive model capable of classifying thefts as **STOLEN** or **RECOVERED**. The project followed a full data science workflow: data exploration, cleaning, feature selection, data modeling, predictive modeling, and evaluation.

Our findings show that bicycle recoveries represent a very small portion of the dataset, creating a highly imbalanced classification problem. After applying data cleaning, encoding, normalization, feature selection, and handling class imbalance with SMOTE-ENN, two machine learning models were trained: Logistic Regression and Decision Tree. Logistic Regression achieved a slightly higher AUC score (0.631) compared to the Decision Tree (0.617), and demonstrated more stable generalization on unseen data.

Therefore, Logistic Regression was selected as the final model for deployment. This model will be packaged using Pickle, deployed with a Flask API, and tested via Postman.

2. Overview of the Solution

The solution follows a structured machine learning pipeline:

1. Data ingestion from CSV.
2. Data exploration using pandas, numpy, matplotlib, and seaborn.
3. Data cleaning, including handling missing values, removing invalid years, correcting outliers, and dropping unnecessary fields.
4. Data preprocessing and modeling, including:
 - Encoding categorical variables
 - Normalizing numeric variables
 - Feature selection (SelectKBest, k=15)
 - Handling imbalanced classes with SMOTE-ENN
5. Predictive modeling using Logistic Regression and Decision Tree.
6. Model evaluation using Accuracy, Confusion Matrices, ROC curves, and AUC.
7. Model deployment using Flask and Pickle.

This pipeline supports reproducible, scalable decision-making around theft recovery prediction.

3. Data Exploration and Findings

3.1. Dataset Summary

- Total rows (raw data): 39,473
- Total columns (raw data): 35
- Total usable rows after cleaning (for the period 2013-2025): 39,460
- Target variable: STATUS (values: STOLEN, RECOVERED)

3.2. Column Types

Numeric features

- Temporal fields: OCC_YEAR, OCC_MONTH, OCC_DAY, OCC_HOUR
- Geographic coordinates: LAT_WGS84, LONG_WGS84
- Bike characteristics: BIKE_COST, BIKE_SPEED
- Administrative codes: division numbers, neighbourhood IDs

Categorical features

- BIKE_TYPE, BIKE_MAKE, BIKE_MODEL, BIKE_COLOUR
- PRIMARY_OFFENCE, LOCATION_TYPE
- Police divisions and neighbourhood names
- Theft attributes such as PREMISES_TYPE, HOOD_158, HOOD_140

3.3. Key Data Quality Findings

3.3.1. Outliers

The raw dataset contained extreme values that would distort model training:

- BIKE_COST
 - Maximum values exceeded \$120,000, which is far above any realistic bicycle price.
 - Strategy: Capped using the 99th percentile.
- BIKE_SPEED
 - Some values were greater than 1,500 km/h, which is physically impossible.
 - Strategy: Outliers replaced with the median speed.

3.3.2. Invalid Data

- Coordinates (LAT/LONG) contained zeros → replaced with NaN then imputed later.
- Negative bike costs were detected → set to median.

3.3.3. Missing Values

Major missingness observed:

Column	Missing Count
BIKE MODEL	14913
BIKE COLOUR	3521
BIKE SPEED	1556
BIKE COST	2604
BIKE MAKE	188

Handling Strategy:

- Categorical: imputed using "Unknown" value
- Numeric: imputed using median

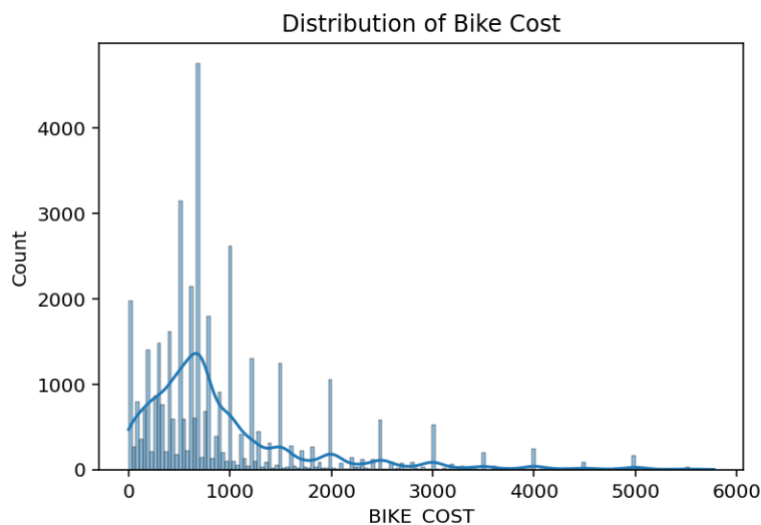
3.3.4. Observed Trends

- Thefts increased steadily from 2016-2020
- Sharp decline from 2021, likely due to COVID-19 lockdowns
- Peak theft hours: 12 to 20 o'clock (afternoons & evenings), with the peak hour at 18
- Theft frequency is strongly seasonal, with the highest incidents occurring in summer months (June-September)
- Higher-value bicycles showed slightly better recovery likelihood, but thefts were most common among mid-priced bikes.
- Location analysis showed that most thefts occurred on streets, parking areas, and outside commercial buildings.
- Recovery rate is extremely low (around 1%), creating a strong class imbalance
→ This directly motivated our use of SMOTE+ENN later.

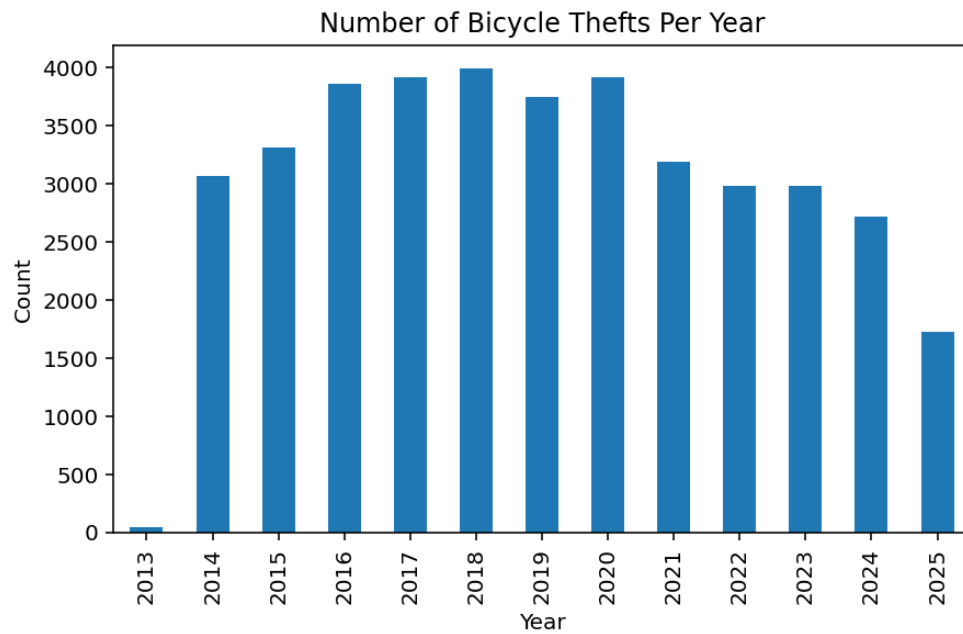
3.4. Visualizations Included

To support our findings, we generated these visualizations using pandas, numpy, matplotlib and seaborn, based on the cleaned data:

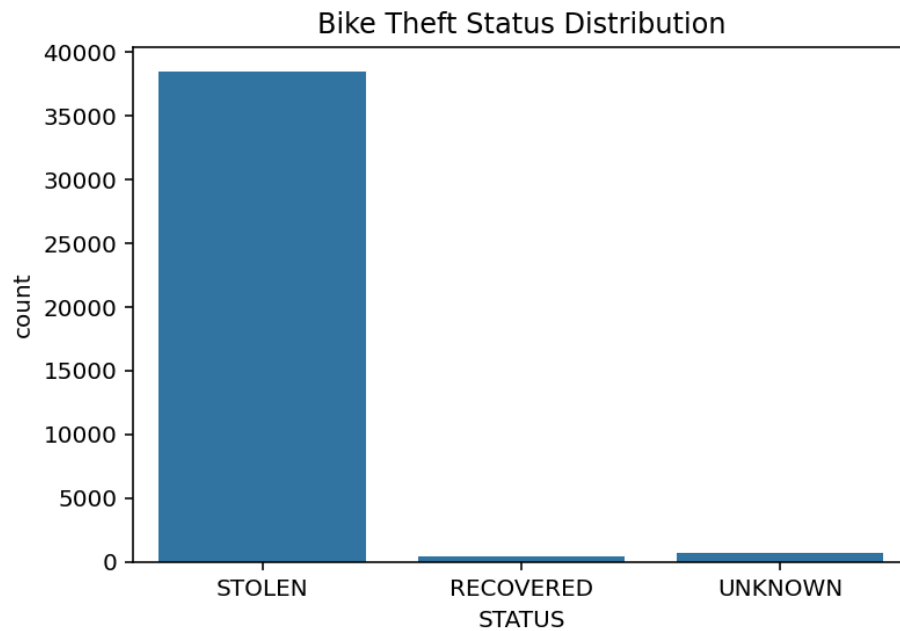
- Distribution of Bicycle Costs



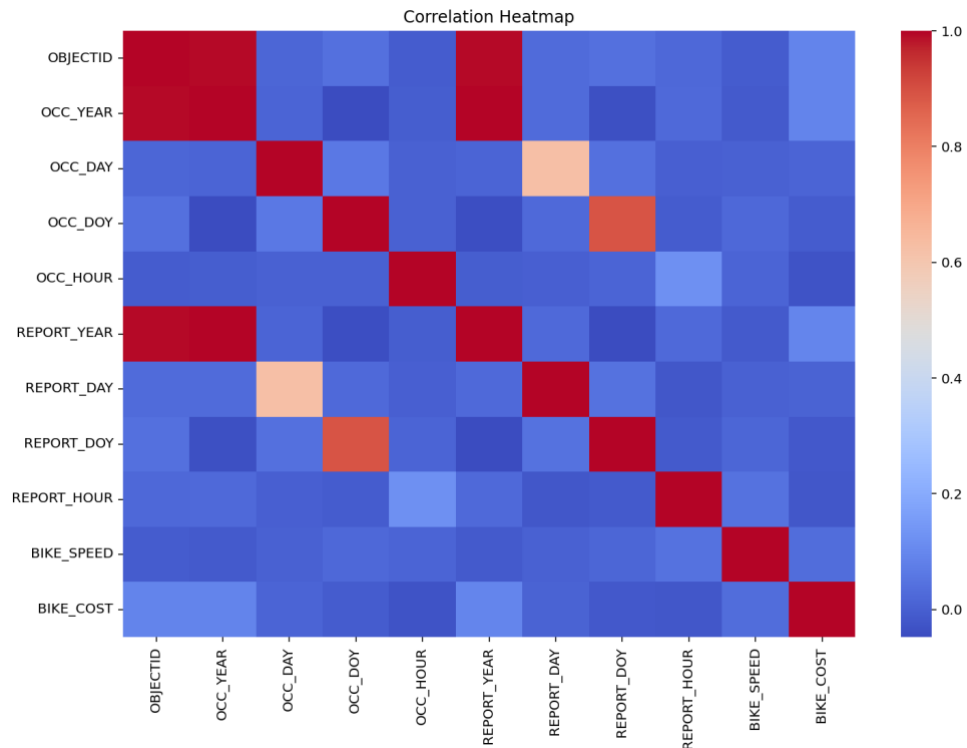
- Thefts per Year / Time-Series Trend



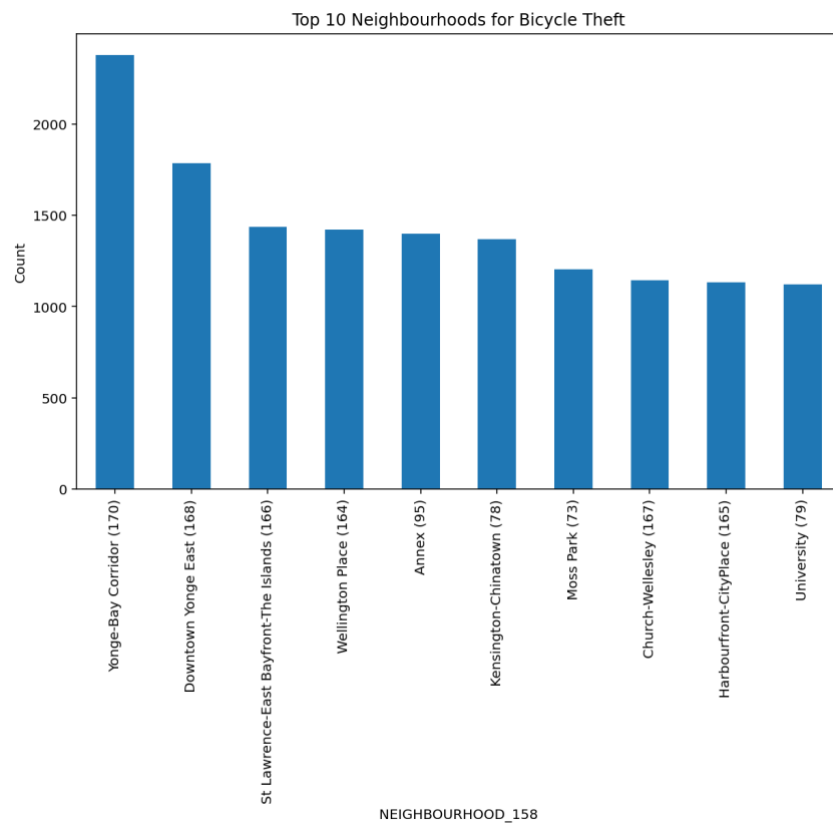
- Theft Status Distribution (STOLEN vs RECOVERED vs UNKNOWN)



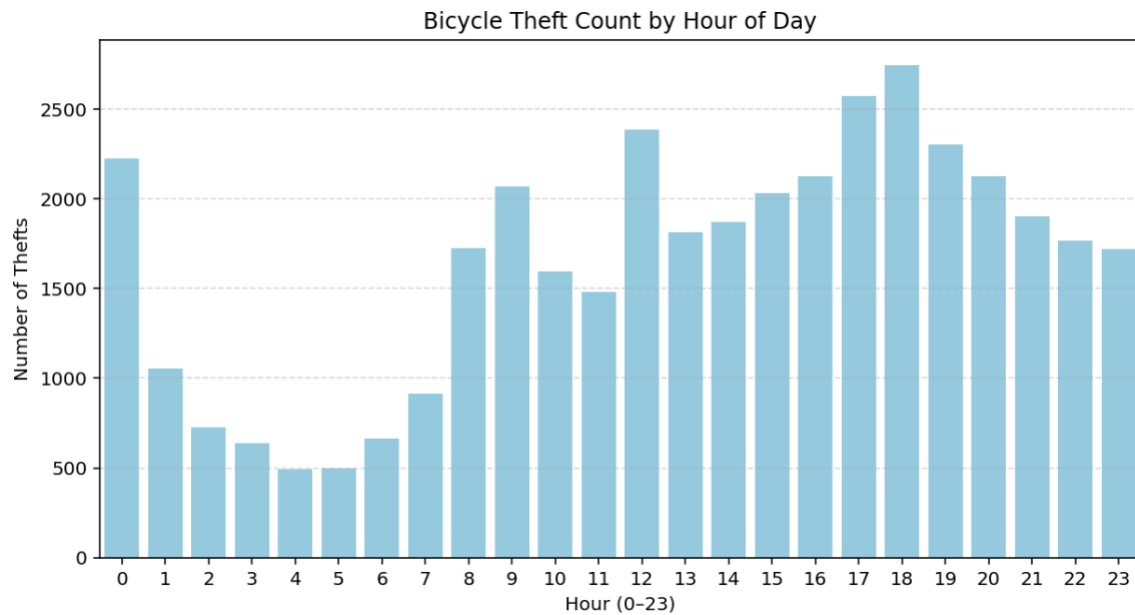
- Correlation Heatmap for numeric fields



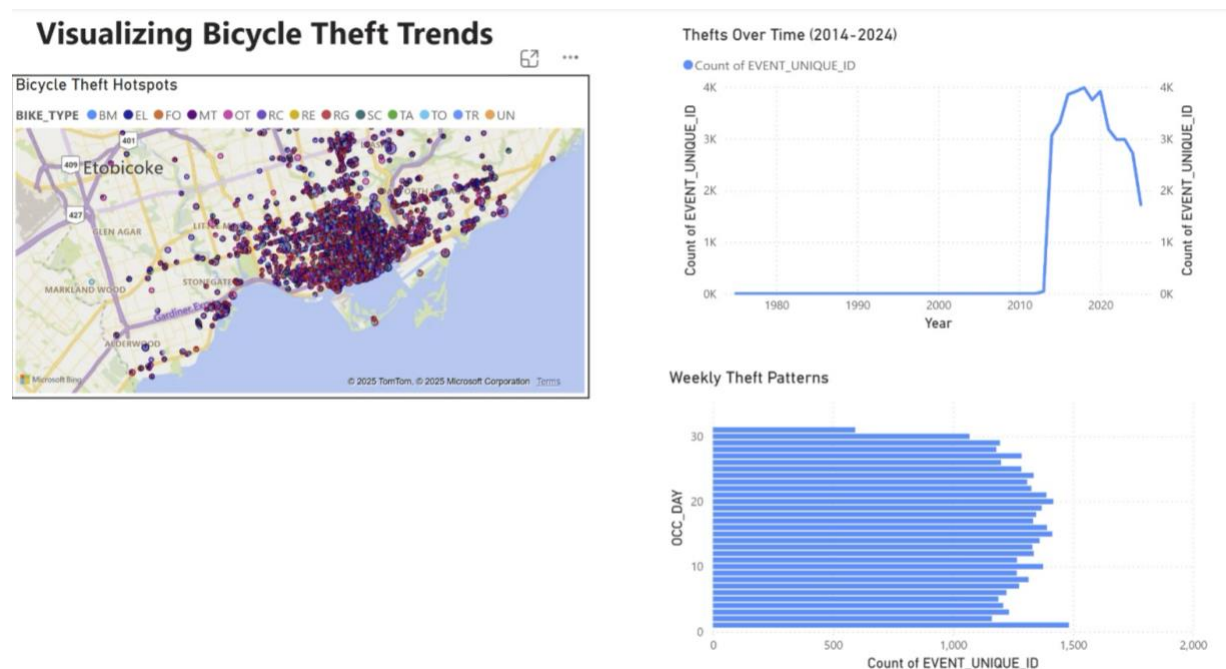
- Top 10 Neighbourhoods with Highest Theft Counts



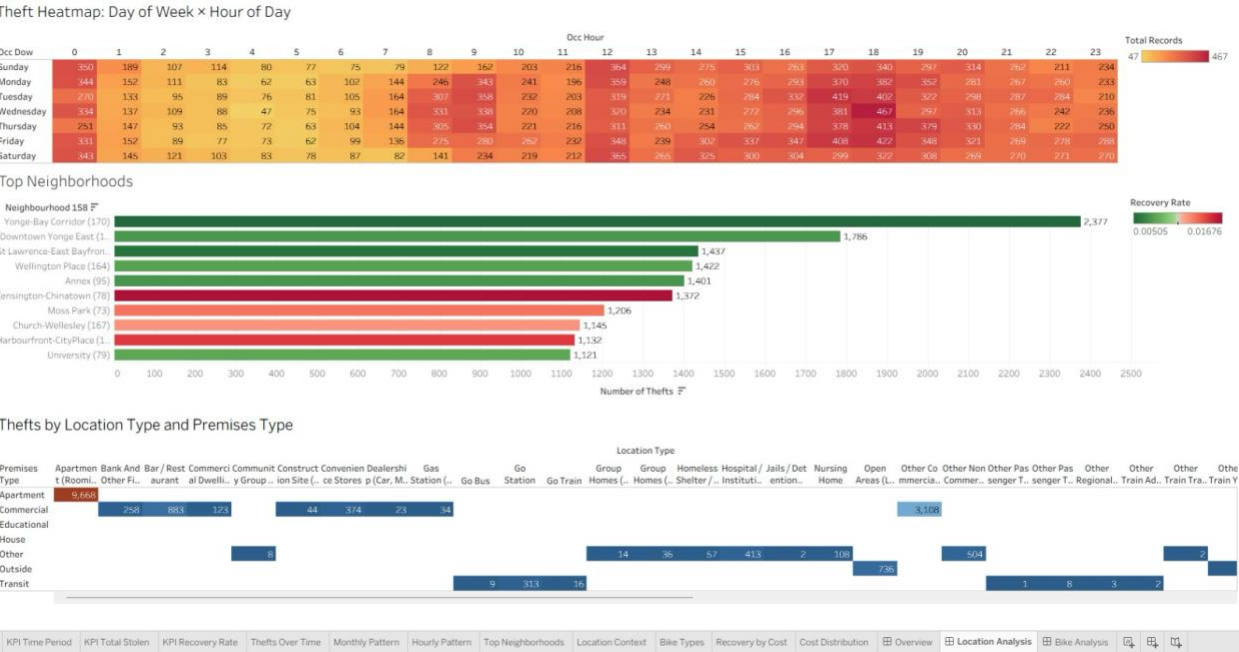
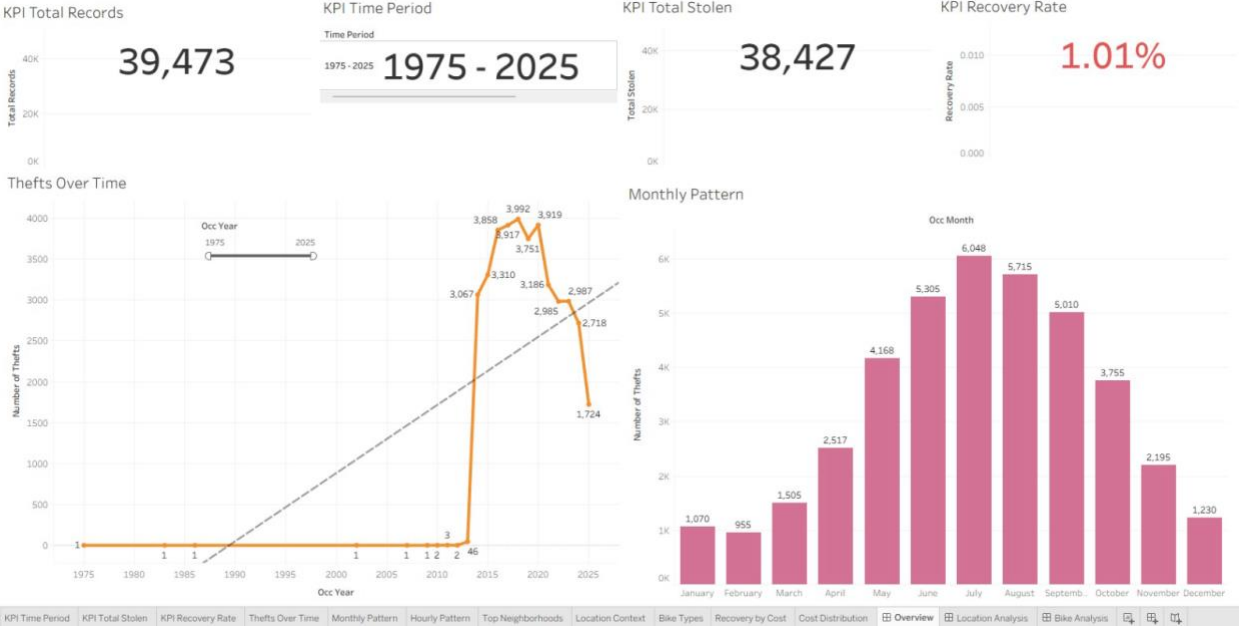
- Bicycle Theft Count by Hour of Day

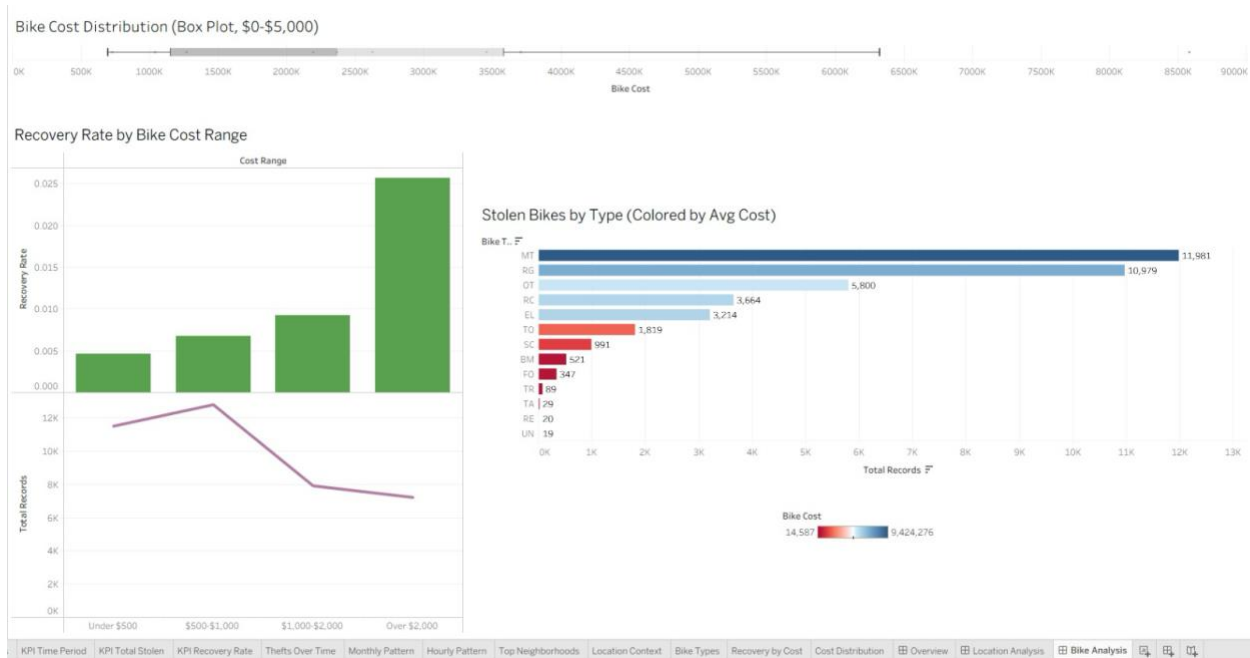


Besides Python, our group also used PowerBI to create an interactive dashboard. The dashboard clearly shows bike theft hotspots in Toronto, the number of bike thefts over time (which reached its peak from 2016 to 2020), as well as the weekly theft patterns.



Another visualization tool we used is Tableau. Overall, Tableau visualizations not only confirmed the patterns identified during Python-based exploration but also highlighted additional insights, especially regarding seasonality, neighbourhood patterns, and the extremely low recovery rate. The following screenshots are the results extracted from our Tableau dashboard.





4. Feature Selection

We used SelectKBest (ANOVA F-test) to evaluate each feature's statistical significance for predicting STATUS.

Steps:

- Preprocessed numeric and categorical data using pipelines.
- Converted preprocessed data into a dense matrix.
- Applied:

```
selector = SelectKBest(score_func=f_classif, k=k_features)
```

- Selected top 15 most informative features.

Results:

F-score analysis revealed that the most predictive features were not bike characteristics, but the PRIMARY_OFFENCE fields recorded by police. In particular, "Property - Found" and "Property - Recovered" had extremely high F-scores (2063 and 388). These fields directly indicate whether a bike was returned or located, meaning they encode information very close to the target variable (STATUS). As a result, they dominate the feature selection process.

Time-based fields such as OCC_YEAR and REPORT_YEAR also showed meaningful predictive power (F-scores around 30), indicating temporal trends in recovery likelihood.

Bike-specific fields such as BIKE_MAKE, BIKE_MODEL, and BIKE_COLOUR had extremely low F-scores and were dropped due to high cardinality and lack of predictive significance.

The screenshot below shows F-scores for all features:

```
=== F-SCORES FOR ALL FEATURES ===
```

	feature	f_score
40	PRIMARY_OFFENCE_PROPERTY - FOUND	2063.185437
39	PRIMARY_OFFENCE_POSSESSION PROPERTY OBC UNDER	845.460222
42	PRIMARY_OFFENCE_PROPERTY - RECOVERED	388.478300
18	PRIMARY_OFFENCE_DRUG - POSS METH (SCHD I)	293.312951
68	PRIMARY_OFFENCE_TRESPASS AT NIGHT	194.833482
36	PRIMARY_OFFENCE_OTHER FEDERAL STATUTE OFFENCES	97.065056
151	LOCATION_TYPE_Other Train Tracks	97.065056
69	PRIMARY_OFFENCE_WEAPON - POSS DANGEROUS PURP	97.065056
24	PRIMARY_OFFENCE_FTC PROBATION ORDER	93.196711
13	PRIMARY_OFFENCE_ASSAULT WITH WEAPON	61.993178
25	PRIMARY_OFFENCE_FTC WITH CONDITIONS	61.993178
47	PRIMARY_OFFENCE_ROBBERY - MUGGING	31.272365
4	REPORT_YEAR	30.441296
0	OCC_YEAR	30.216095
63	PRIMARY_OFFENCE_THEFT UNDER - BICYCLE	26.797233
62	PRIMARY_OFFENCE_THEFT UNDER	25.273774
48	PRIMARY_OFFENCE_ROBBERY - OTHER	24.926785
162	LOCATION_TYPE_Streets, Roads, Highways (Bicycl...	24.011752
37	PRIMARY_OFFENCE_POSSESSION HOUSE BREAK INSTRUM	22.724684
170	LOCATION_TYPE_Ttc Subway Train	14.506793
11	PRIMARY_OFFENCE_ASSAULT	12.164142
178	PREMISES_TYPE_Outside	11.981016
173	PREMISES_TYPE_Apartment	10.112404
126	LOCATION_TYPE_Apartment (Rooming House, Condo)	10.112404
119	DIVISION_D43	9.580207

Features retained include:

- Time-Based Features
 - OCC_YEAR – Year when the bicycle theft occurred
 - REPORT_YEAR – Year when the theft was reported
- Primary Offence Features (Highly Predictive)
 - PRIMARY_OFFENCE_ASSAULT WITH WEAPON
 - PRIMARY_OFFENCE_DRUG - POSS METH (SCHD I)
 - PRIMARY_OFFENCE_FTC PROBATION ORDER
 - PRIMARY_OFFENCE_FTC WITH CONDITIONS
 - PRIMARY_OFFENCE_OTHER FEDERAL STATUTE OFFENCES
 - PRIMARY_OFFENCE_POSSESSION PROPERTY OBC UNDER
 - PRIMARY_OFFENCE_PROPERTY - FOUND
 - PRIMARY_OFFENCE_PROPERTY - RECOVERED
 - PRIMARY_OFFENCE_ROBBERY - MUGGING
 - PRIMARY_OFFENCE_THEFT UNDER - BICYCLE
 - PRIMARY_OFFENCE_TRESPASS AT NIGHT
 - PRIMARY_OFFENCE_WEAPON - POSS DANGEROUS PURP
- Location Feature
 - LOCATION_TYPE_Other Train Tracks

The screenshot belows show the selected features for the model:

```
=== SELECTED FEATURES (TOP-K) ===  
OCC_YEAR  
REPORT_YEAR  
PRIMARY_OFFENCE_ASSAULT WITH WEAPON  
PRIMARY_OFFENCE_DRUG – POSS METH (SCHD I)  
PRIMARY_OFFENCE_FTC PROBATION ORDER  
PRIMARY_OFFENCE_FTC WITH CONDITIONS  
PRIMARY_OFFENCE_OTHER FEDERAL STATUTE OFFENCES  
PRIMARY_OFFENCE_POSSESSION PROPERTY OBC UNDER  
PRIMARY_OFFENCE_PROPERTY – FOUND  
PRIMARY_OFFENCE_PROPERTY – RECOVERED  
PRIMARY_OFFENCE_ROBBERY – MUGGING  
PRIMARY_OFFENCE_THEFT UNDER – BICYCLE  
PRIMARY_OFFENCE_TRESPASS AT NIGHT  
PRIMARY_OFFENCE_WEAPON – POSS DANGEROUS PURP  
LOCATION_TYPE_Other Train Tracks
```

5. Data Modeling

This section describes how the cleaned dataset was transformed, prepared, and structured for machine learning. It details the data cleaning strategy, wrangling techniques, assumptions & constraints, feature selection process, and the preparation of the final training/testing datasets.

5.1 Data Cleaning Strategy

5.1.1. Filtering Valid Years (2013-2025)

Records outside this range were removed to maintain consistency with recent reporting standards and to reduce noise from older or incomplete records.

5.1.2. Correcting Invalid or Unrealistic Values

- Negative BIKE_COST values → replaced with the median cost.
- Extreme outliers above the 99th percentile → capped to median.
- BIKE_SPEED values exceeding realistic physical limits (e.g., >120 km/h) → replaced with median.
- Invalid coordinates (0.0 latitude/longitude) → replaced with NaN.

5.1.3. Handling Missing Data

- Categorical fields (e.g., BIKE_MAKE, BIKE_MODEL, BIKE_COLOUR) → filled with "Unknown"
- Numeric fields → filled with median, which is robust against outliers.

5.1.4. Removing High-Cardinality or Non-Predictive Fields

Some fields introduced noise, high dimensionality, or acted as identifiers rather than features:

- High-cardinality bike attributes: BIKE_MODEL, BIKE_MAKE, BIKE_COLOUR

- Identifier or redundant fields: EVENT_UNIQUE_ID, OBJECTID, x, y

These were excluded prior to model training.

5.1.5. Results of Data Cleaning

After cleaning:

- Final usable rows: 38,811
- Remaining columns after dropping irrelevant ones: 20
- STATUS distribution remained highly imbalanced:
 - STOLEN $\approx$ 98.9%
 - RECOVERED $\approx$ 1.1%

This confirmed the need for resampling later in the pipeline.

5.2 Data Wrangling Techniques

To ensure consistent and machine-readable input:

- Target Variable Definition: STATUS was restricted to two meaningful outcomes:
 - RECOVERED $\rightarrow$ 0
 - STOLEN $\rightarrow$ 1
 - Rows with other status values were filtered out.
- Type Standardization
 - Date/time features (year, month, hour) were retained in numeric form.
 - Categorical police fields (offence, division, location type, premises type) were kept as strings for encoding.

5.3 Assumptions and Constraints

Assumptions:

- Missing bike details (Make, Model, Colour) do not meaningfully improve return prediction.
- Coordinates with value 0.0 are invalid and cannot be meaningfully imputed.
- Police offence codes are strong predictors of case outcomes.
- Using 2013-2025 avoids biases from procedural changes in earlier years.

Constraints:

- Severe class imbalance limited model performance until resampling was applied.
- Some neighbourhood fields in raw data were inconsistent or duplicated, so they were excluded.
- High-cardinality attributes were removed because one-hot encoding would create hundreds of features.

5.4 Train-Test Split

We split the data into training and testing sets using stratified sampling to preserve the imbalance ratio:

- 70% → X_train, y_train
- 30% → X_test, y_test
- test_size=0.3, random_state=42, stratify=y

This ensures that both STOLEN and RECOVERED classes are represented similarly in both sets and that evaluation happens on previously unseen data.

5.5 Preprocessing Pipelines

We used scikit-learn's ColumnTransformer and Pipeline to define separate preprocessing steps for numeric and categorical features.

Numeric Pipeline

```
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
```

- Missing numeric values → replaced with the median.
- Features → scaled to [0, 1] using MinMaxScaler.

Categorical Pipeline

```
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
```

- Missing categorical values → replaced with the most frequent category.
- Categorical variables → encoded using OneHotEncoder with handle_unknown="ignore" to avoid errors when unseen categories appear in the test set.

Both pipelines are combined into:

```
preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, numeric_cols),
    ("cat", categorical_pipeline, categorical_cols)
])
```

The preprocessor is fitted on the training data and later applied to both train and test sets.

5.6 Feature Selection

After preprocessing, we applied univariate feature selection:

```
k_features = 15
```

```
selector = SelectKBest(score_func=f_classif, k=k_features)
X_train_selected = selector.fit_transform(X_train_processed, y_train)
```

- We used ANOVA F-test (f_classif) to score each feature's relationship with the target.
- The top 15 features were retained (k_features=15), reducing dimensionality and focusing the model on the most informative predictors.
- The same preprocessor + selector combination is then applied to the test set (X_test → X_test_processed → X_test_selected).

Some transformers produce sparse matrices, so we safely converted them to dense arrays when necessary.

5.7 Handling Class Imbalance

The dataset is highly imbalanced, with STOLEN cases vastly outnumbering RECOVERED cases. To address this, we used SMOTEENN on the training set:

- SMOTE synthesizes new minority (RECOVERED) samples.
- ENN (Edited Nearest Neighbours) removes noisy or ambiguous majority samples.

This combination both oversamples RECOVERED and cleans STOLEN points near the decision boundary.

At the end of this stage, we have:

- X_train_resampled, y_train_resampled → balanced, feature-selected training data
- X_test_selected_dense, y_test → untouched evaluation data

6. Predictive Model Building and Evaluation

We implemented and compared two supervised classification models using scikit-learn: Logistic Regression, Decision Tree Classifier.

Both models are trained on the resampled training set and evaluated on the original test set.

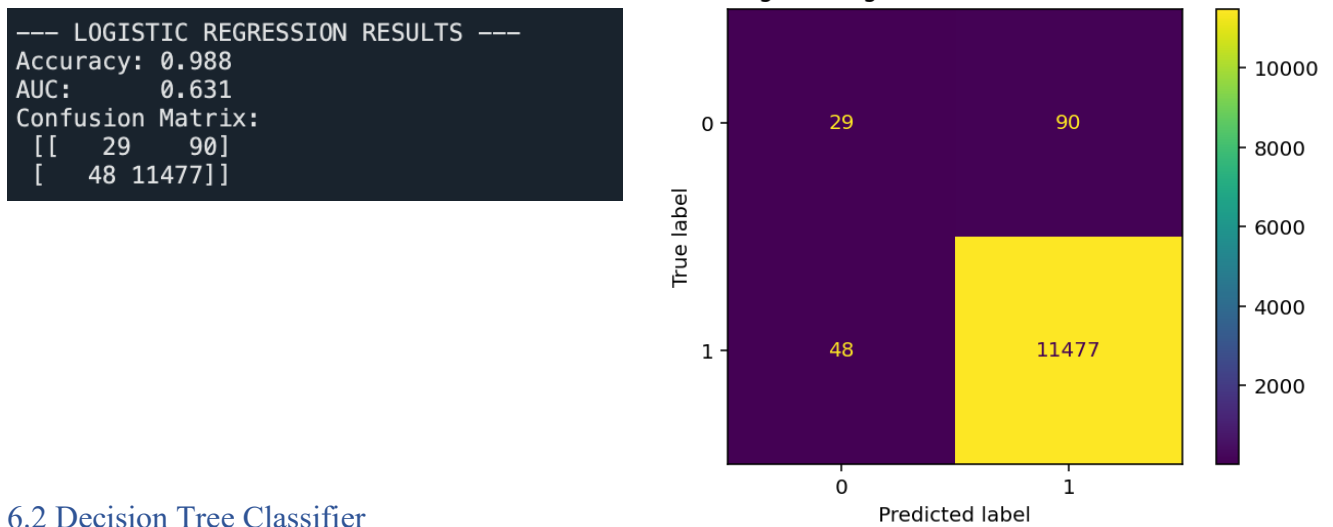
6.1 Logistic Regression

Logistic Regression is a linear model that estimates the probability of a bike being STOLEN (1) vs RECOVERED (0). max_iter=1000 ensures convergence after scaling and resampling.

We then computed:

- Predicted labels: log_preds
- Predicted probabilities: log_probs
- Accuracy on the test set
- Confusion matrix using confusion_matrix + ConfusionMatrixDisplay
- ROC AUC using roc_auc_score on the predicted probabilities

A confusion matrix plot for Logistic Regression is generated and shown in the report:



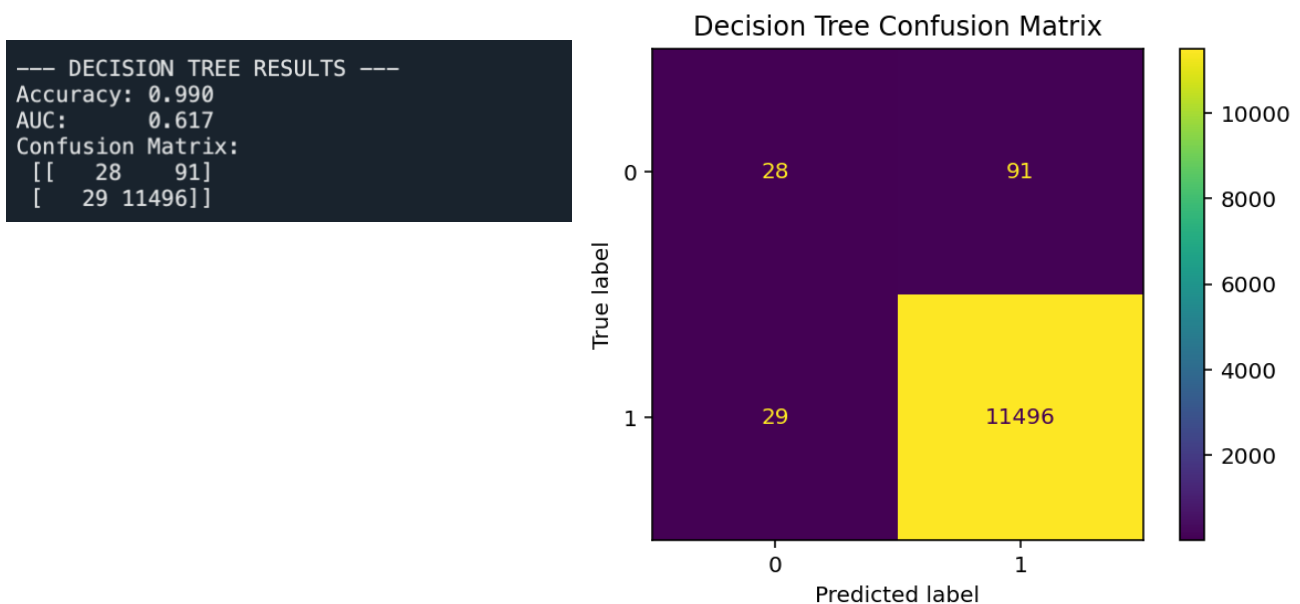
6.2 Decision Tree Classifier

A Decision Tree with `max_depth=6` was chosen to limit overfitting and keep the tree interpretable.

For the tree model we computed:

- Predicted labels: `tree_preds`
- Predicted probabilities: `tree_probs`
- Accuracy, confusion matrix, and ROC AUC using the same metrics as above.

A second confusion matrix is plotted for the Decision Tree:



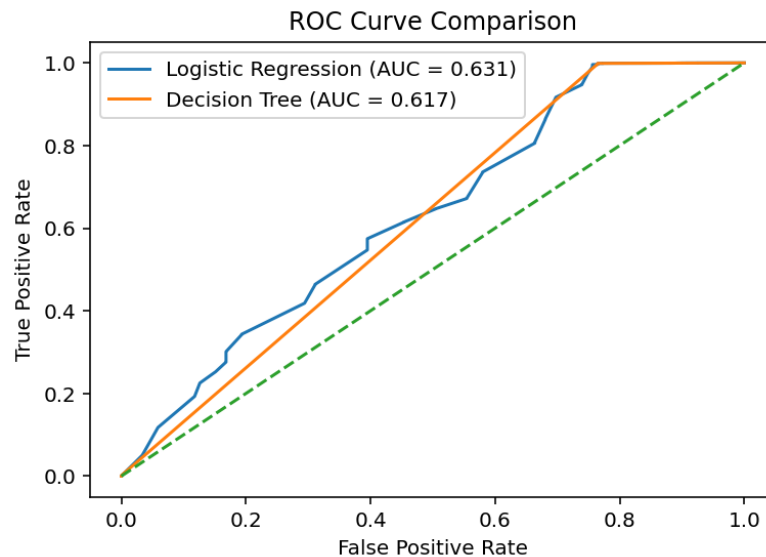
6.3 ROC Curve Comparison

To visually compare the two models, we plotted both ROC curves on the same figure:


```
fpr1, tpr1, _ = roc_curve(y_test, log_probs)
fpr2, tpr2, _ = roc_curve(y_test, tree_probs)

plt.plot(fpr1, tpr1, label=f"Logistic Regression (AUC = {log_auc:.3f})")
plt.plot(fpr2, tpr2, label=f"Decision Tree (AUC = {tree_auc:.3f})")
plt.plot([0, 1], [0, 1], linestyle="--")
```

- The diagonal line represents random guessing.
- Curves further above this diagonal indicate better discrimination between STOLEN and RECOVERED.
- AUC values for both models are printed in the legend.



6.4 Best Model Selection

Finally, we selected the best model based on ROC AUC:

```
if log_auc > tree_auc:
    best_model = "Logistic Regression"
else:
    best_model = "Decision Tree"
```

The script prints:

```
BEST PERFORMING MODEL: Logistic Regression
```

Logistic regression model will be used in the deployment step.

7. Deployment of the Model

In this phase, the objective was to operationalize the selected predictive model by deploying it as a real-time API service. This enables external applications such as web clients to submit bicycle theft incident data and receive a prediction on whether the bicycle is likely to be recovered.

Our deployment pipeline follows all requirements: Flask API, model serialization using pickle, and client testing using both HTML and JSON requests via tools like Postman.

7.1. Model Serialization

Before deployment, the final trained model and its preprocessing objects were saved to disk using Python's pickle module:

- Preprocessor (imputation, scaling, encoding)
- Feature selector (SelectKBest)
- Final classifier (Logistic Regression - best performing model)
- Feature list

7.2. Flask-Based API Service

We built a RESTful web service using Flask, defined in the file app.py

Main Endpoints:

- GET/: Returns a simple health check response: "Bike Return Prediction API is running."
- POST/predict: Accepts JSON input and returns machine learning predictions.

Returns:

- prediction_label ("RETURNED" or "NOT RETURNED (stolen)")
- prediction_class (0 or 1)
- probability_not_returned (float between 0–1)

7.3. Preprocessing and Prediction Flow

Inside the API, incoming JSON is processed using this sequence (same as training):

1. Load JSON into pandas DataFrame
2. Ensure all 19 expected fields exist
3. Apply preprocessing pipeline
4. Apply SelectKBest feature selector
5. Convert sparse matrix → dense (if necessary)
6. Generate probability scores using Logistic Regression
7. Return prediction + probability

This ensures complete consistency between training and inference.

7.4. HTML User Interface

To support interactive testing, two frontend endpoints were added:

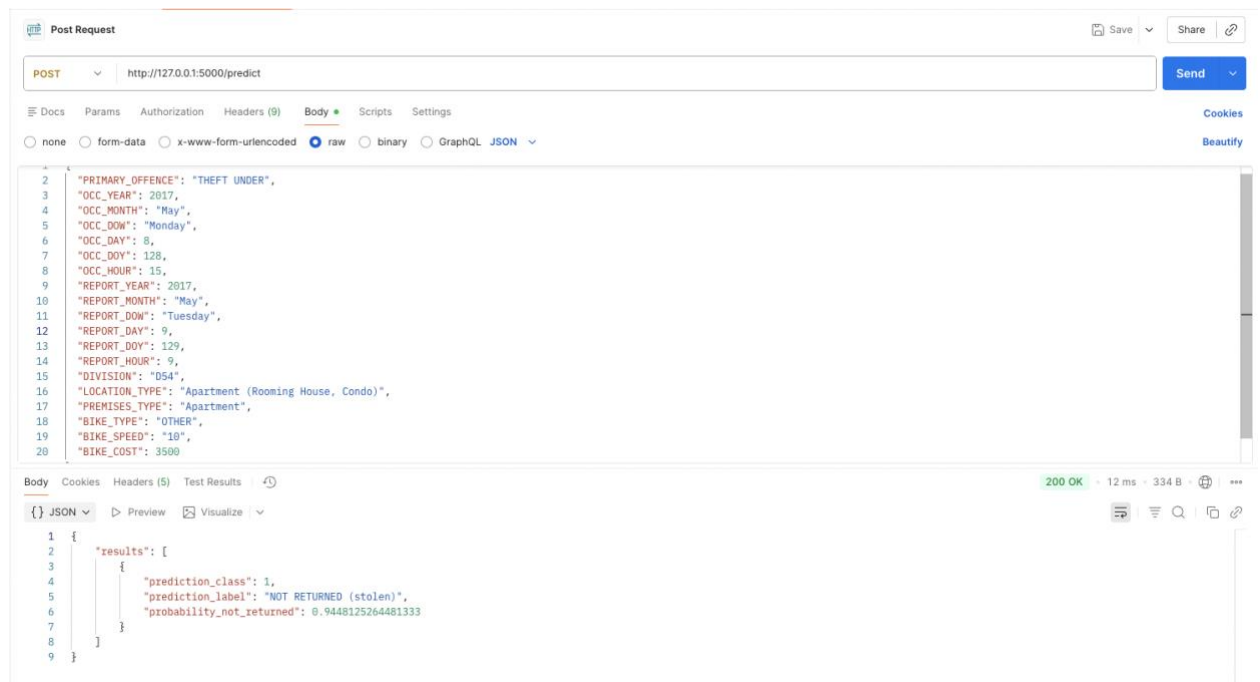
- GET/ui: Displays an HTML form (bike_form.html) that allows the user to manually enter incident attributes.
- POST/ui/result: Processes form input and returns a prediction page (bike_result.html).

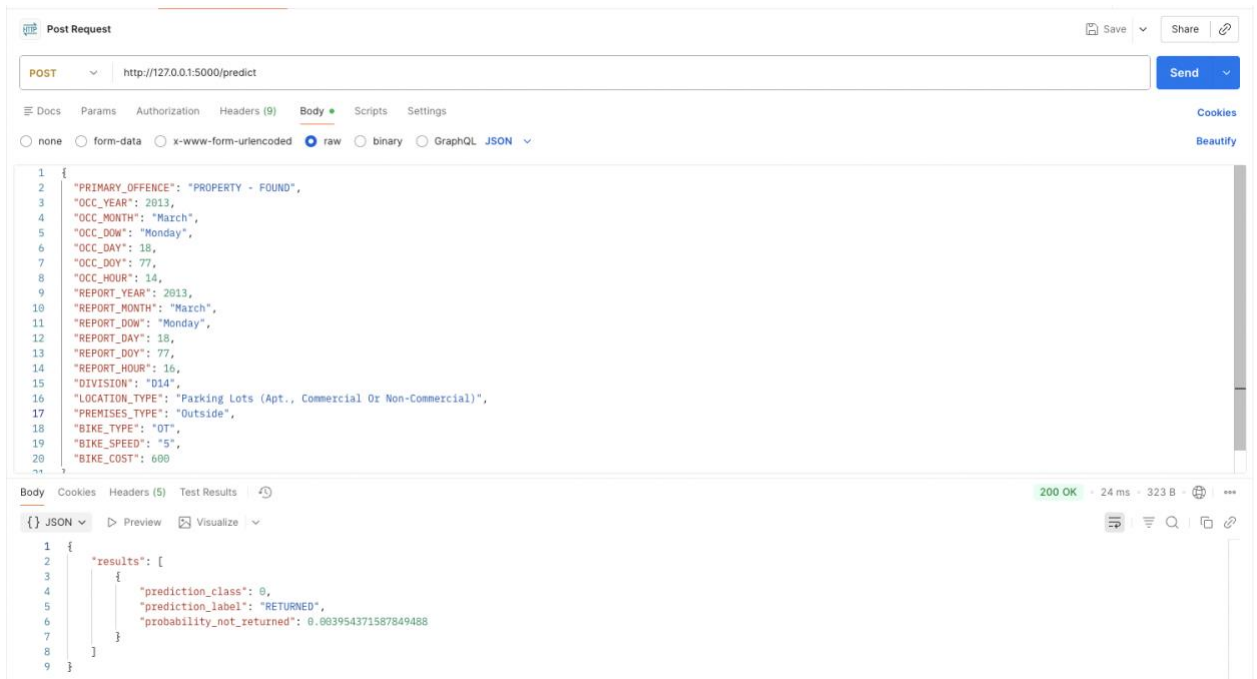
7.5. Testing the API (Postman + UI)

We validated the API using:

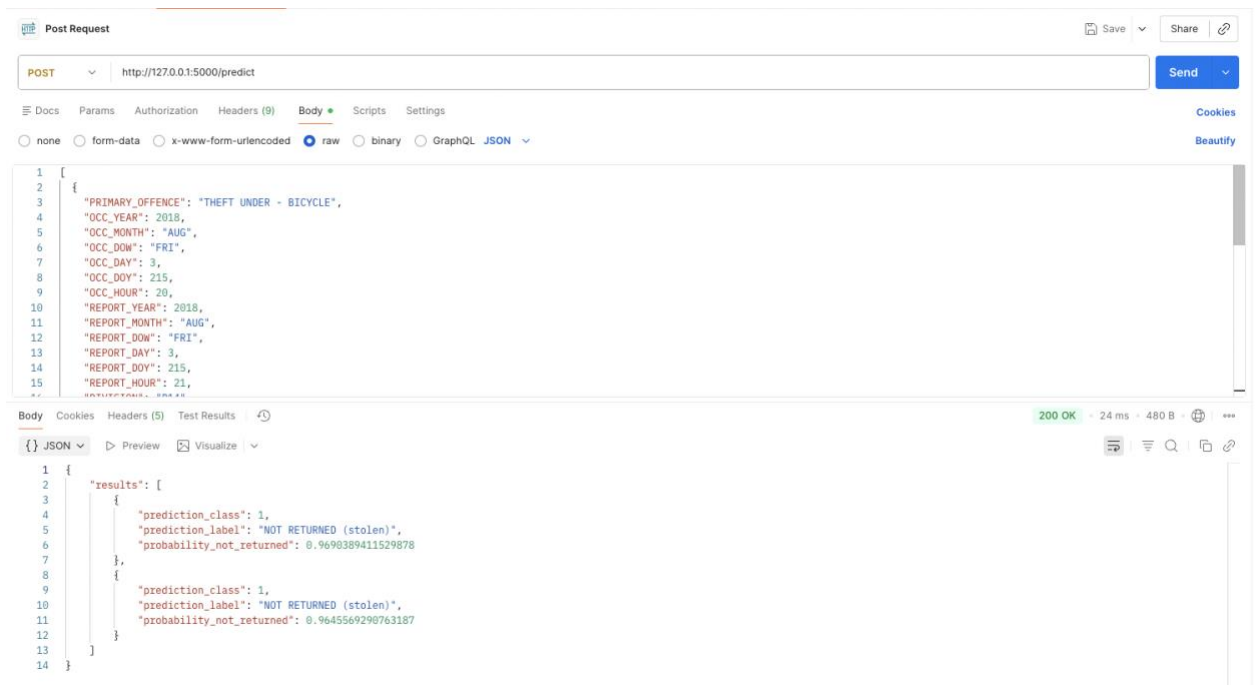
- Postman Client API
 - Sent POST requests with sample records
 - Verified predictions and probability scores
 - Tested error handling (missing fields, wrong types)

The screenshot below shows 2 sample POST requests for a basic single prediction with one “NOT RETURNED” result and one “RETURNED” result:





The screenshot below shows that our API handles list input:



The screenshot below shows the API still works with missing numeric field (BIKE_SPEED, for example):

Post Request

POST http://127.0.0.1:5000/predict

Body

```
1 {
2   "PRIMARY_OFFENCE": "THEFT UNDER - BICYCLE",
3   "OCC_YEAR": 2017,
4   "OCC_MONTH": "SEP",
5   "OCC_DOW": "THU",
6   "OCC_DAY": 21,
7   "OCC_DOY": 264,
8   "OCC_HOUR": 18,
9   "REPORT_YEAR": 2017,
10  "REPORT_MONTH": "SEP",
11  "REPORT_DOW": "THU",
12  "REPORT_DAY": 21,
13  "REPORT_DOY": 264,
14  "REPORT_HOUR": 19,
15  "DIVISION": "051",
16  "LOCATION_TYPE": "Outside",
17  "PREMISES_TYPE": "Outside",
18  "BIKE_TYPE": "Hybrid",
19  "BIKE_COST": 750
20 }
```

Body

JSON

```
1 {
2   "results": [
3     {
4       "prediction_class": 1,
5       "prediction_label": "NOT RETURNED (stolen)",
6       "probability_not_returned": 0.9710692090554487
7     }
8   ]
9 }
```

The screenshot below shows the API can handle missing categorical field (for example, LOCATION_TYPE):

Post Request

POST http://127.0.0.1:5000/predict

Body

```
1 {
2   "PRIMARY_OFFENCE": "THEFT UNDER - BICYCLE",
3   "OCC_YEAR": 2016,
4   "OCC_MONTH": "MAY",
5   "OCC_DOW": "WED",
6   "OCC_DAY": 4,
7   "OCC_DOY": 125,
8   "OCC_HOUR": 14,
9   "REPORT_YEAR": 2016,
10  "REPORT_MONTH": "MAY",
11  "REPORT_DOW": "WED",
12  "REPORT_DAY": 4,
13  "REPORT_DOY": 125,
14  "REPORT_HOUR": 15,
15  "DIVISION": "011",
16  "PREMISES_TYPE": "House",
17  "BIKE_TYPE": "Mountain Bike",
18  "BIKE_SPEED": 17,
19  "BIKE_COST": 650
20 }
```

Body

JSON

```
1 {
2   "results": [
3     {
4       "prediction_class": 1,
5       "prediction_label": "NOT RETURNED (stolen)",
6       "probability_not_returned": 0.9729700559690994
7     }
8   ]
9 }
```

- Web Browser UI
 - Submitted form values
 - Confirmed correct preprocessing and predictions
 - Validated full request→response cycle

The screenshot below shows the UI with prefilled data, ready to test:

The first screenshot shows the 'Bike Return Prediction' form in a web browser. The form is titled 'Bike Return Prediction' and includes a subtitle: 'Enter the incident details below to predict whether the bike is likely to be returned or not returned (stolen)'. The form fields are as follows:

- Primary Offence:** THEFT UNDER
- Division:** D14
- Location Type:** Outside
- Premises Type:** Commercial
- Occurrence (OCC) Date & Time:**
 - OCC Year: 2018
 - OCC Month: July
 - OCC Day of Week: Monday
 - OCC Day: 16
 - OCC Day of Year: 197
 - OCC Hour: 14
- Report Date & Time:**
 - Report Year: 2018
 - Report Month: July
 - Report Day of Week: Monday
 - Report Day: 16
 - Report Day of Year: 197
 - Report Hour: 15
- Bike Details:**
 - Bike Type: MOUNTAIN
 - Bike Speed: 10
 - Bike Cost (\$): 800

Below the form fields is a blue 'Predict' button. At the bottom of the form, a small note states: 'This page sends your input to the same model API used in Postman: POST /predict.'

The second screenshot shows the 'Prediction Result' page. The page title is 'Prediction Result'. The prediction result is displayed in a box:

NOT RETURNED (stolen)
 Prediction class: 1
 Probability bike NOT returned (stolen): 94.11%

Below the prediction result is a section titled 'Input Details' which shows the JSON payload sent to the API:

```
{
  "BIKE_COST": 800,
  "BIKE_SPEED": 10,
  "BIKE_TYPE": "MOUNTAIN",
  "DIVISION": "D14",
  "LOCATION_TYPE": "Outside",
  "OCC_DAY": 16,
  "OCC_DOW": "Monday",
  "OCC_DOW": 197,
  "OCC_HOUR": 14,
  "OCC_MONTH": "July",
  "OCC_YEAR": 2018,
  "PREMISES_TYPE": "Commercial",
  "PRIMARY_OFFENCE": "THEFT UNDER",
  "REPORT_DAY": 16,
  "REPORT_DOW": "Monday",
  "REPORT_DOW": 197,
  "REPORT_HOUR": 15,
  "REPORT_MONTH": "July",
  "REPORT_YEAR": 2018
}
```

At the bottom of the 'Input Details' section is a blue link: '← Back to form'.

7.6. Deployment Output Summary

Artifacts produced:

- bike_return_model.pkl (serialized model bundle)
- app.py (Flask application API)
- bike_form.html and bike_result.html (user interface templates)
- Successful API server running locally at: <http://127.0.0.1:5000>

Core accomplishments:

- Fully functional prediction API
- Consistent preprocessing between training and deployment
- Supports both programmatic and manual testing
- Produces clear, interpretable outputs

END OF REPORT